



CSE6364

SOFTWARE EVOLUTION &
MAINTENANCE

GROUP PROJECT

PROJECT TITLE: EXPENSE TRACKER APP

GROUP NO.: 3

NG JIN MUN
YU TING HUI
LIM HUAN YEE

241UC240BF
241UC240ZD
241UC240BH

DATE: 26/04/2026

TABLE OF CONTENTS

Table of Contents	1
1 Introduction.....	2
1.1 System Overview	3
1.1 Objective of the Proposal.....	3
2 Problem Statement	3
3 Justification for improvement.....	4
3.1 Rationale for Selection	5
3.2 Expected Outcomes	5
4 Proposed Solution	5
4.1 Description of Changes	6
4.1.1 Login & Logout Authentication	6
4.1.2 Multi-Currency Support with Auto-Conversion	6
4.1.3 Expense Categories with Budget Goals	6
4.1.4 Spending Graphs and Charts	6
4.1.5 Expense Search and Filter	6
4.1.6 Recurring Expenses	6
4.1.7 Progressive Web App with Offline Mode.....	7
4.2 Implementation Plan.....	7
4.3 Tools and Frameworks	8
5 Impact Analysis.....	9
5.1 Software Impact	10
5.2 Risk Management.....	10
6 Timeline	11
6.1 Task Breakdown	12
6.2 Gantt chart	12
7 Conclusion	13
8 References.....	14
9 Appendices	15

1 INTRODUCTION

1.1 SYSTEM OVERVIEW

The selected project is the **Expense Tracker App** by ManikMaity (n.d.). Currently, it is a lightweight, frontend-focused web application designed to help users log their daily financial transactions. The existing system provides basic functionalities such as adding, deleting, and calculating total income and expenses. However, it relies entirely on the browser's local Storage for data retention and lacks backend architecture. While the current codebase is clean and functional for temporary use, it operates as an isolated, single-device tool rather than a modern, scalable financial application.

1.2 OBJECTIVE OF THE PROPOSAL

The primary objective of this project is to apply software evolution and maintenance principles to transform the existing static Expense Tracker into a robust, cloud-enabled, and highly analytical Progressive Web App (PWA). By integrating modern frameworks and backend services (such as Firebase Auth and live APIs), this proposal aims to resolve the system's current architectural limitations. Our goal is to deliver a production-ready application that offers secure multi-user access, cross-device data synchronization, and advanced financial insights.

2 PROBLEM STATEMENT

Despite its clean user interface, the current version of the Expense Tracker App suffers from several critical software limitations that prevent it from being a reliable tool for long-term personal finance management. We have identified three major problem areas in the current codebase:

1. **Data Volatility and Lack of Privacy (Architectural Flaw):** The most critical issue is the reliance on local Storage. If a user clears their browser cache or loses their device, all financial history is permanently deleted. Furthermore, the absence of an authentication system means the application cannot support multiple users; anyone who opens the app on a shared device has full access to the financial data.
2. **Absence of Analytical Tools and Budgeting Constraints (Functional Flaw):** Currently, the system only displays a flat list of transactions and a basic net balance. It fails to provide meaningful financial insights. Users cannot categorize their spending (e.g., Food, Transport), nor can they visualize their financial habits over time. Without visual charts or budget limit alerts, the software acts as a simple calculator rather than a true financial management tool.
3. **Poor Usability for Real-World Scenarios (Usability Flaw):** The application lacks modern conveniences expected from financial software. It does not support multiple currencies, making it unusable for travel or international users. Additionally, users cannot search for past transactions, filter by date, or automate recurring bills (like monthly subscriptions). Finally, because it is a standard web page, users cannot easily log expenses offline or install it natively on their mobile devices for quick access.

3 JUSTIFICATION FOR IMPROVEMENT

3.1 RATIONALE FOR SELECTION

We selected this specific project and these enhancements because they represent a scenario for "Software Evolution." Moving a system from a local, single-user environment to a cloud-synced, multi-user architecture is highly relevant to modern industry practices (Sommerville, 2015).. Instead of just fixing minor UI bugs, our proposed changes fundamentally upgrade the system's architecture and value. Implementing features like Firebase Authentication, REST API integration (for live currency rates), and Chart.js for data visualization will allow our team to demonstrate a comprehensive understanding of full-stack development, API handling, and database integration within a realistic 14-week timeline. The current codebase is manageable enough to understand quickly, but immature enough to provide a massive surface area for meaningful, high-impact contributions.

3.2 EXPECTED OUTCOMES

Upon successful implementation of our proposed enhancements, the evolved software will deliver the following outcomes:

- **Reliability & Security:** Through Firebase backend integration, users will have secure login credentials, ensuring their financial data is private and safely synchronized across all their devices (desktop and mobile).
- **Actionable Financial Insights:** The integration of category tagging and Chart.js will provide users with visual spending trends (pie charts and line graphs). The new budget goals feature will actively warn users when they are nearing their monthly limits.
- **Enhanced User Experience:** The addition of a search/filter system and recurring expense automation will drastically reduce the time users spend managing their data.
- **Global & Mobile Accessibility:** By implementing a live currency conversion API and upgrading the platform to a Progressive Web App (PWA), the software will be usable offline, installable on mobile home screens, and fully adaptable for international use.

4 PROPOSED SOLUTION

4.1 DESCRIPTION OF CHANGES

To address the three core problem areas identified in the current Expense Tracker App — data volatility and lack of privacy, absence of analytical tools, and poor usability for real-world scenarios — seven targeted enhancements have been proposed. Each change directly resolves a specific architectural, functional, or usability flaw in the existing system, and together they transform the application into a secure, cloud-enabled, and analytically rich financial management tool.

4.1.1 Login & Logout Authentication

Firebase Authentication will replace the current localStorage model, enabling secure email/password login and cloud-hosted data storage via Firestore. Expense records will persist across devices and survive browser cache clears. A logout button will allow users on shared devices to safely end their session, transforming the app into a secure, multi-user, cloud-synchronized platform.

4.1.2 Multi-Currency Support with Auto-Conversion

A live exchange rate API will be integrated so users can log expenses in any currency (e.g. USD, MYR, EUR) and have amounts automatically converted to their chosen base currency. Each transaction will display both the original and converted amounts, eliminate manual conversion and make the app globally usable.

4.1.3 Expense Categories with Budget Goals

Users will be able to tag transactions under categories such as Food, Transport, or Bills, and set monthly spending limits for each. A real-time progress bar will track spending against each budget, turning red as the limit approaches to serve as a visual warning and encourage active financial oversight.

4.1.4 Spending Graphs and Charts

Using Chart.js, a line chart will visualize spending trends over time while a pie/donut chart will break down expenditure by category. Both charts update dynamically as transactions are added, turning raw records into actionable financial insights.

4.1.5 Expense Search and Filter

A robust search and filter interface will be implemented to improve data navigability within the transaction history. Users will be able to perform keyword searches to find specific vendors or items and apply filters based on categories, price ranges, or specific date intervals. This functionality ensures that as the volume of logged data grows over time, users can quickly retrieve specific records and manage their historical spending with high efficiency.

4.1.6 Recurring Expenses

Users can mark expenses as recurring (weekly or monthly), and the system will automatically log them at the set interval. Browser notifications will also remind users of upcoming recurring charges, reducing manual effort and improving day-to-day reliability.

4.1.7 Progressive Web App with Offline Mode

A service worker and web app manifest will be added to meet PWA standards, allowing users to install the app on their home screen like a native app. Expenses can be added and viewed offline, with all data automatically synced to Firestore once connectivity is restored.

4.2 IMPLEMENTATION PLAN

The seven enhancements will be implemented in a phased and incremental manner, ensuring that each feature is built upon a stable foundation and integrated cleanly without disrupting existing functionality. All development will be conducted on dedicated feature branches before merging into the main codebase.

Phase 1 — Firebase Authentication & Database Migration

Firebase will be initialized with full auth flows (register, login, logout) using the Firebase Authentication SDK. Existing localStorage logic will then be refactored to read/write from Firestore, with records organized by user ID to ensure data privacy between accounts.

Phase 2 — Category Budgeting & Spending Graphs

Category tagging and per-category monthly budget limits will be added to the transaction form, with data stored in Firestore. A color-coded progress bar will track spending against each budget. Chart.js will then be integrated to render a line chart for spending trends and a pie/donut chart for category breakdown on a dedicated analytics dashboard.

Phase 3 — Multi-Currency Conversion & Recurring Expenses

A live exchange rate API will be connected, with currency selection on the transaction form and auto-conversion on submission. Both original and converted amounts will be stored and displayed. A recurring expense engine will also be built, allowing users to define transactions with a set frequency,

backed by a browser notification system for reminders and a background scheduler for automatic logging.

Phase 4 — Search, Filter & PWA Setup

A search and filter interface will be built over the transaction list, supporting keyword, category, and date range queries with preset time period shortcuts. A service worker and web app manifest will then be added for asset caching, offline support, and home screen installation. Offline entries will be temporarily stored in IndexedDB and synced to Firestore once connectivity is restored.

Phase 5 — Integration Testing & Documentation

Once all seven features are in place, the complete integrated system will undergo functional validation. Edge cases such as failed API calls, expired authentication tokens, offline data conflicts during synchronization, and recurring expense scheduling errors will be tested and handled with appropriate error responses. The project's README, user guide, and inline code comments will be updated to cover all new features, Firebase setup instructions, API key management, and PWA installation steps.

4.3 TOOLS AND FRAMEWORKS

The following tools and frameworks have been selected based on their compatibility with the existing frontend codebase, their maturity and community support, and their suitability for the specific requirements of each proposed enhancement.

- **Firebase Authentication & Cloud Firestore** — A Google-provided Backend-as-a-Service platform that handles email/password login and cloud data storage for all transactions, budgets, and recurring expense configurations, with built-in security rules ensuring each user can only access their own data (Google, n.d.).
- **ExchangeRate-API** — A REST-based service that fetches live exchange rates on each foreign currency submission, with the API key secured via environment variables to prevent exposure in the public repository.
- **Chart.js** — A lightweight, framework-agnostic JavaScript charting library that renders the spending trend line chart and category breakdown pie/donut chart directly into the existing frontend without requiring a full framework migration.
- **Service Worker & Web App Manifest (PWA)** — Enables asset caching, offline data entry via IndexedDB, background sync to Firestore upon reconnection, and home screen installation through a native Service Worker and JSON manifest file.

- **Browser Notifications API** — Delivers recurring expense reminder alerts to users, with permission requested on first use and notifications scheduled based on due dates stored in Firestore.
- **Git & GitHub** — Manages version control with feature branches isolating each enhancement and pull requests enforcing peer review before any changes are merged into the main branch.

5 IMPACT ANALYSIS

5.1 SOFTWARE IMPACT

The proposed solutions will greatly transform the Expense Tracker App from a basic utility into a robust, secure, and user-centric financial management tool. The following are the impacts of the solutions:

1. Functional Transformation
 - By integrating the Firebase Authentication into the Expense Tracker App, it ensures that every user has a private and secure environment for their financial data.
 - By implementing the recurring expenses feature, it helps reduce manual data entry, improving the system's utility and promoting long-term user retention.
 - By adding Category Budgeting and Multi-Currency Auto-Conversion, it provides real-time feedback on spending limits and global currency impacts.
2. Technical & Architectural Impact
 - The codebase will be expanded to include external dependencies such as Chart.js for visualization and live exchange rate APIs for data accuracy.
 - Standardized security protocols for handling user credentials and private spending records will be introduced once Firebase is implemented.
 - The adoption of PWA functionality modifies the app's architecture to handle data synchronization to allow it to function reliably regardless of the network connectivity.
3. User Experience (UX) & Accessibility
 - The Spending Graphs transform raw text data into visual insights, making it easier for users to identify their spending patterns.
 - The Search and Filter mechanism makes navigating large transaction histories easier, which improves user efficiency.
 - By making the Expense Tracer App portable on mobile devices, it increases accessibility and ease of use
4. Documentation & Maintenance Impact
 - The project's README, installation Guide, and User Manual will be updated to include instructions for Firebase setup, API key management, and PWA installation.

5.2 RISK MANAGEMENT

To make sure the evolution can be done successfully, the following potential risks and established mitigation strategies are identified:

Table 5.1: Risk Management

Risk Category	Risk Description	Potential Impact	Mitigation Strategy
Technical risk	Integration issues with Firebase Authentication and external APIs	Features may not function correctly or even cause system crashes	Conduct early integration testing and implement error-handling mechanisms
Security risk	Exposure of API keys or improper handling	Data breaches or unauthorized access to user financial data	Store API keys securely using environment variables, use Firebase

	of user authentication data		security rules, and implement HTTPS encryption
Performance risk	Increased load due to real-time data fetching and PWA synchronization	Slower application performance	Optimize API calls, use caching strategies, and implement efficient data synchronization methods
Data accuracy risk	Inaccurate or outdated exchange rates from third-party APIs	Incorrect financial calculations, which may lead to misleading users	Use reliable APIs and implement periodic updates
Usability risk	Complex UI due to added features like graphs, filters, and budgeting	Users may feel overwhelmed when using the app	Apply user-centred design and keep UI simple
Maintenance risk	Increased complexity of the codebase due to additional features and dependencies	Difficult to debug, update, or maintain the system	Maintain proper documentation and follow coding standards
User data loss risk	Failure in data synchronization or storage	Loss of user financial records	Implement backup mechanisms and auto-sync features

6 TIMELINE

6.1 TASK BREAKDOWN

This project is divided into four primary phases to ensure a systematic approach to software maintenance and evolution.

- Phase 1: Proposal Development (Week 1 – Week 4)
 - Analyse the Expense Tracker App's existing structure, functionality, and areas for potential improvement.
 - Review documentation to identify bugs and performance bottlenecks.
 - Draft the proposal, including introduction, problem statement, justification for improvement, proposed solution, impact analysis, timeline, and conclusion.
- Phase 2: Implementation of Solutions (Week 5 – Week 10)
 - Implement the enhancements, ensuring all modifications are clean, maintainable, and integrate seamlessly with the existing system.
- Phase 3: Validation & Reporting (Week 11 – Week 12)
 - Validate the functionality of all new features to make sure the improvements do not raise new issues.
 - Update the README, provide user guides for new features, and add inline comments for clarity.
- Phase 4: Presentation (Week 13 – Week 14)
 - Prepare the presentation to showcase improvements made on the Expense Tracker App and explain their impact.

Table 6.1 Task Distribution

Phase	Task description	Member responsible
Phase 1	Introduction, Problem Statement, Justification for Improvement	Lim Huan Yee
	Proposed Solution, Conclusion	Yu Ting Hui
	Impact Analysis, Timeline	Ng Jin Mun
Phase 2	Firebase Auth, Budgeting Logic, and Search & Filter	Ng Jin Mun
	Multi-currency API, Integration, and Recurring Expenses Engine	Yu Ting Hui
	Chart.js Data Visualization and PWA Setup	Lim Huan Yee
Phase 3	Functional validation and documenting test results	All members
	Update README, User Guides, and inline comments	All members
Phase 4	Prepare the presentation	All members

6.2 GANTT CHART

The following Gantt chart shows our group's plan for improving the Expense Tracker app. By splitting our work into four clear phases, we ensure all the tasks can be completed on time.

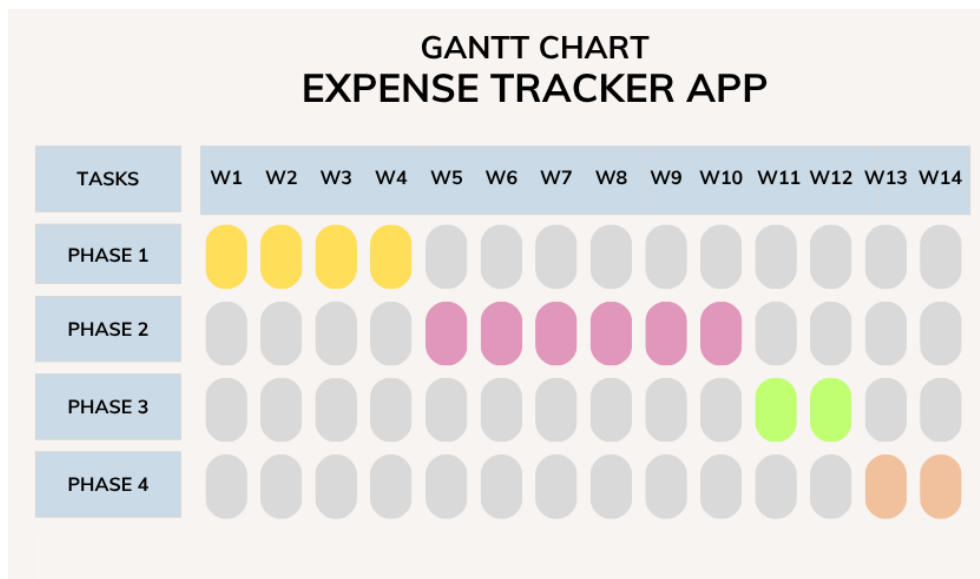


Figure 6.1: Gantt chart of expense tracker app

The Expense Tracker App by ManikMaity, while commendable as a clean and functional frontend utility, currently operates with three significant limitations that prevent it from serving as a dependable long-term personal finance tool. Its sole reliance on `localStorage` makes financial records vulnerable to loss when cache is cleared or devices are switched. The absence of authentication prevents data privacy on shared devices, and the lack of analytical tools, multi-currency support, and transaction automation limits its real-world practicality.

These limitations make this project a highly suitable candidate for software evolution, targeting fundamental architectural and usability flaws rather than surface-level fixes. The seven proposed enhancements address each problem systematically. Firebase Authentication and Firestore resolve data volatility and privacy at an architectural level. Multi-currency conversion and recurring expense automation close the real-world usability gaps. Category budgeting with progress bars and Chart.js visualizations transform the app from a passive calculator into an active financial management tool. Search and filter ensure long-term navigability, while the PWA upgrade with offline sync eliminates internet dependency and enables native-like installation. Potential risks across security, performance, and usability have been identified with clear mitigations including environment variable key management, early integration testing, and user-centred design practices.

Upon completion, the Expense Tracker App will no longer be an isolated single-device tool. It will be a secure, cloud-synchronized, and analytically capable Progressive Web App that meets the expectations of a modern user managing personal finances across devices, currencies, and connectivity conditions — serving as a practical demonstration of how a structurally sound but limited codebase can be systematically rearchitected to meet real-world standards of quality, scalability, and user experience.

8 REFERENCES

Google. (n.d.). *Firebase documentation.* <https://firebase.google.com/docs>

Maity, M. (n.d.). *Expense Tracker App* [Computer software]. GitHub.

Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson.

